

Vehicle Rental System

A Console-Based Object-Oriented Programming Project in Java

Course	Object-Oriented Programming with Java
Project Type	Console-Based Mini Project
Student Name	Sarthak Jena
Roll No. / Reg. No.	25BAI10247

Keywords: Java, Object-Oriented Programming, Inheritance, Polymorphism, Abstraction, Encapsulation, Interfaces, Exception Handling, Console Application

Table of Contents

1. Introduction
2. Problem Statement
3. Functional Requirements
4. Non-Functional Requirements
5. System Architecture
6. Design Diagrams
 - 6.1 Use Case Diagram
 - 6.2 Workflow Diagram
 - 6.3 Sequence Diagram
 - 6.4 Class / Component Diagram
 - 6.5 ER Diagram (Not Applicable)
7. Design Decisions & Rationale
8. Implementation Details
9. Screenshots / Results
10. Testing Approach
11. Challenges Faced
12. Learnings & Key Takeaways
13. Future Enhancements
14. References

1. Introduction

This report documents the design, implementation, and testing of a **Vehicle Rental System** — a menu-driven, console-based Java application built as a mini project for a second-year Object-Oriented Programming (OOP) course. The application allows an operator to manage a small vehicle fleet consisting of Cars, Bikes, and Trucks, register customers, process vehicle rentals and returns, and generate rental cost estimates — all through a simple text-based menu.

The project's primary academic purpose is to demonstrate, in one coherent and logically connected codebase, every major pillar of Object-Oriented Programming: classes and objects, encapsulation, inheritance, abstraction, polymorphism, method overloading, method overriding, constructors, interfaces, the Java Collections framework (ArrayList), and exception handling. Rather than showing these concepts as isolated snippets, the project ties them together in a working, testable application, which is the approach this report follows throughout.

The system is implemented entirely in core Java using the standard library only — no external frameworks, databases, or GUI toolkits are used, in line with the scope expected of an introductory OOP assignment.

2. Problem Statement

Vehicle rental agencies need a reliable way to track which vehicles they own, which ones are currently available, who has rented them, and how much each rental costs. Doing this manually — on paper or in scattered spreadsheets — is slow and error-prone: it is easy to double-book a vehicle that is already rented out, lose track of who has returned what, or make mistakes calculating rent for different vehicle types that each have their own pricing rules (a car is not priced the same way as a truck carrying cargo).

This project addresses that problem by building a small, self-contained software system that manages a vehicle fleet, its customers, and its rental transactions in one place. The system enforces business rules automatically — a vehicle cannot be rented twice at once, a vehicle that was never rented cannot be "returned," and rental cost is calculated consistently based on vehicle type and duration — removing the risk of manual error.

Beyond the practical use case, the project also serves an academic purpose: it is designed as a learning exercise to demonstrate the core principles of Object-Oriented Programming through a realistic, working application rather than isolated textbook examples.

Target Users

- **Students and instructors** evaluating the project as an academic OOP assignment.
- **Small rental-shop operators or staff** who need a lightweight tool to track vehicle availability and rentals without commercial rental-management software.
- **Developers/learners** studying Java OOP who want a complete, runnable reference that ties abstraction, inheritance, polymorphism, interfaces, and exception handling together in one codebase.

3. Functional Requirements

The system shall provide the following operations through a menu-driven console interface:

ID	Requirement	Description
FR-1	Add Vehicle	Register a new Car, Bike, or Truck with its type-specific attributes (seats/AC, engine capacity, or load capacity). Duplicate vehicle IDs must be rejected.
FR-2	Display Vehicles	List the entire fleet, or filter to show only vehicles currently available for rent.
FR-3	Search Vehicle	Locate vehicles by ID, by brand and model, or by a maximum daily rental rate (three overloaded search operations).
FR-4	Register Customer	Capture and store customer ID, name, phone number, and driving licence number.
FR-5	Rent Vehicle	Assign an available vehicle to a registered customer for a specified number of days and automatically compute the total cost.
FR-6	Return Vehicle	Close an active rental and mark the vehicle as available again.
FR-7	Calculate Rental Cost	Estimate rental cost in advance for a given vehicle and duration, with an optional discount percentage.
FR-8	Display Customer Details	Show all registered customers in tabular form.
FR-9	Display Rental Details	Show the full rental history, including which rentals are ongoing and which are returned.
FR-10	Exit	Terminate the program safely.
FR-11	Input Validation	Reject and re-prompt on invalid numeric input rather than crashing the program.
FR-12	Business Rule Enforcement	Prevent renting an already-rented vehicle, prevent returning a vehicle that was never rented, require at least 1 rental day, and require a discount between 0 and 100 percent.

4. Non-Functional Requirements

Category	Requirement
Usability	The menu must be simple, numbered, and self-explanatory so a non-technical operator can use it without training.
Reliability	The application must never terminate abruptly due to bad input or a broken business rule; all failures must be caught and reported as readable messages.
Maintainability	Code must be organised into small, single-responsibility classes (one class per file) so that new vehicle types or operations can be added with minimal changes elsewhere.
Portability	The application must run on any platform with a standard JDK (Windows, macOS, Linux) with no external dependencies beyond the Java standard library.

Category	Requirement
Performance	All operations must complete instantly for the expected data volumes of a small fleet (tens to low hundreds of vehicles/customers/rentals) using in-memory ArrayList storage.
Extensibility	The design must allow new vehicle types to be added by writing one new subclass of Vehicle, without modifying existing classes (Open/Closed Principle).
Data Integrity	The system must prevent inconsistent states — e.g. a vehicle cannot be simultaneously marked available and have an open rental record.

Note: Persistent storage, concurrency, security/authentication, and networked/multi-user access are explicitly out of scope for this version (see Section 13, Future Enhancements).

5. System Architecture

The application follows a simple layered architecture that separates user interaction from business logic and from the domain model. This separation means `Main` never stores data directly, and `VehicleRentalSystem` never reads from the keyboard — each class has exactly one reason to change.

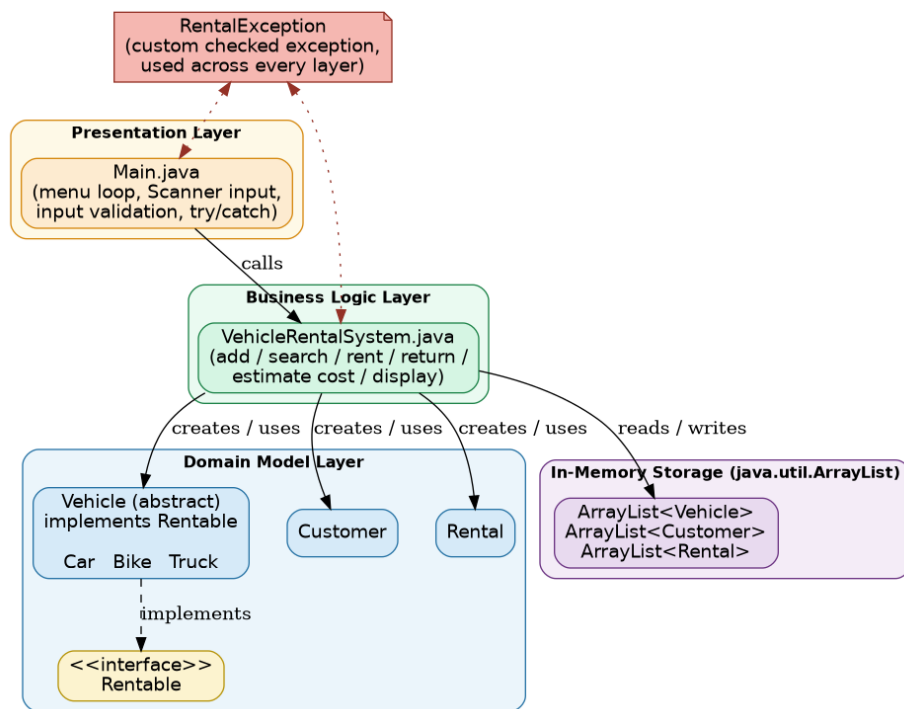


Figure 5.1 — Layered architecture of the Vehicle Rental System.

Layer	Class(es)	Responsibility
Presentation	Main.java	Displays the menu, reads user input with a Scanner, validates numeric input, and routes each menu choice to the appropriate operation. Catches and reports exceptions.
Business Logic	VehicleRentalSystem.java	Owns the three ArrayLists and implements every operation: adding/searching vehicles, registering/searching customers, renting, returning, and estimating cost. Enforces business rules and throws RentalException when a rule is broken.
Domain Model	Vehicle, Car, Bike, Truck, Customer, Rental, Rentable	Represents the real-world entities and their behaviour: vehicle pricing rules, customer data, and the link between a customer, a vehicle, and a rental period.
Storage	java.util.ArrayList (in VehicleRentalSystem)	Holds all run-time data in memory for the duration of the program.

6. Design Diagrams

6.1 Use Case Diagram

The Operator is the single actor in this system. Every menu option corresponds directly to one use case, reflecting the straightforward, single-user nature of a console application.

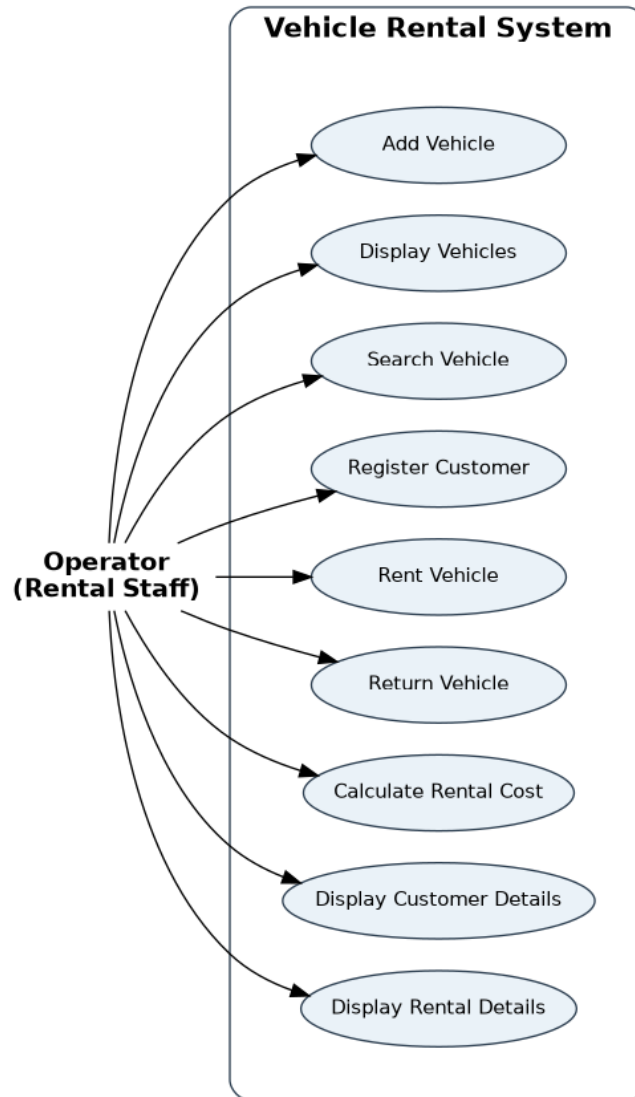


Figure 6.1 — Use Case Diagram: Operator interacting with the system.

6.2 Workflow Diagram

The workflow diagram below captures the main program loop: display the menu, read a choice, validate it, execute the chosen operation inside a try/catch block, and return to the menu — repeating until the user exits.

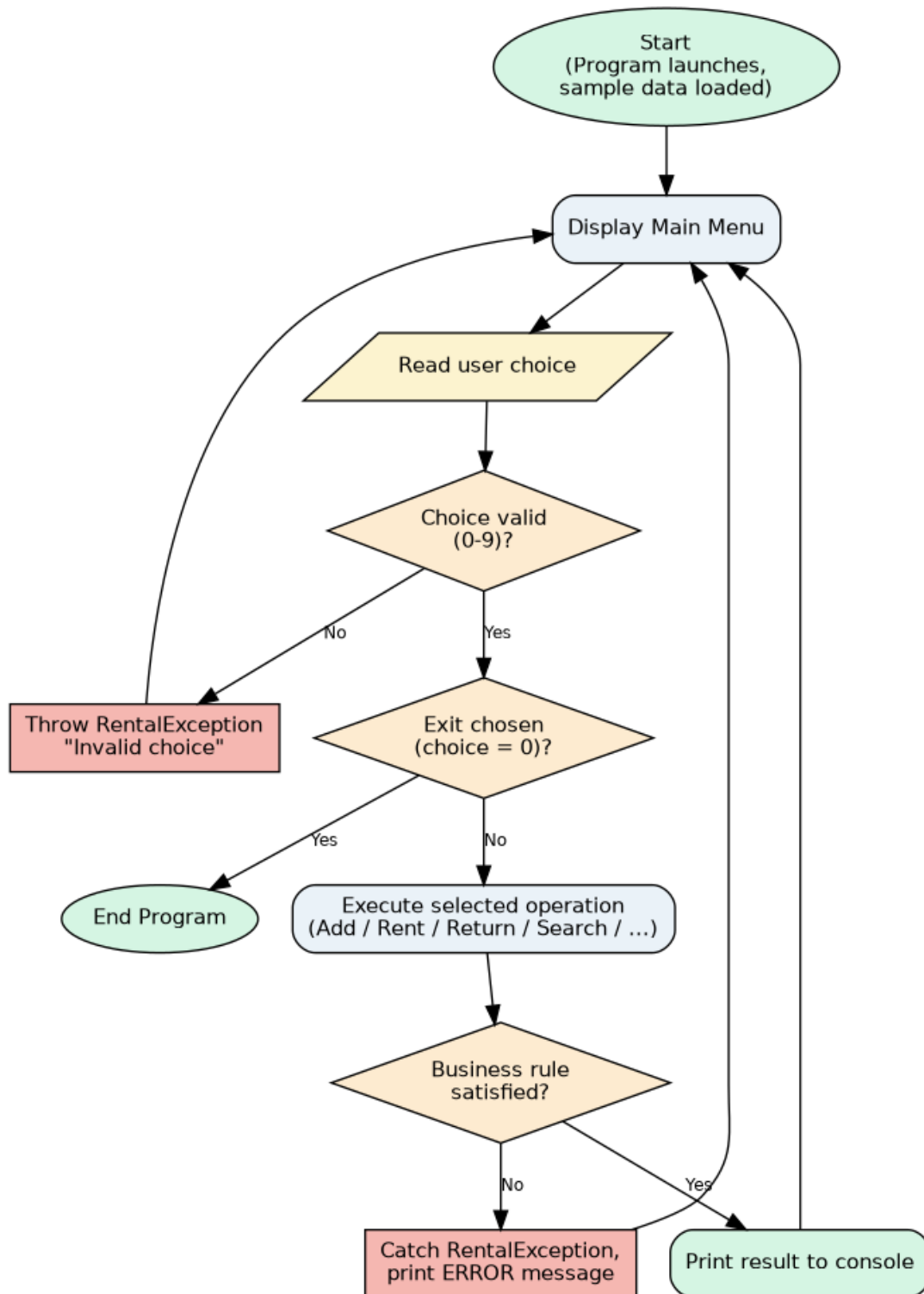


Figure 6.2 — Overall program workflow, including the exception-handling path.

6.3 Sequence Diagram

This sequence diagram traces the **Rent Vehicle** use case end to end: the customer and vehicle are looked up, the vehicle's availability is flipped through the Rentable interface, a Rental object is created (which internally calls the polymorphic, overridden `calculateRentalCost()`), and the confirmation is returned to the user. The alt-box at the bottom shows the exception path taken when a business rule is violated.

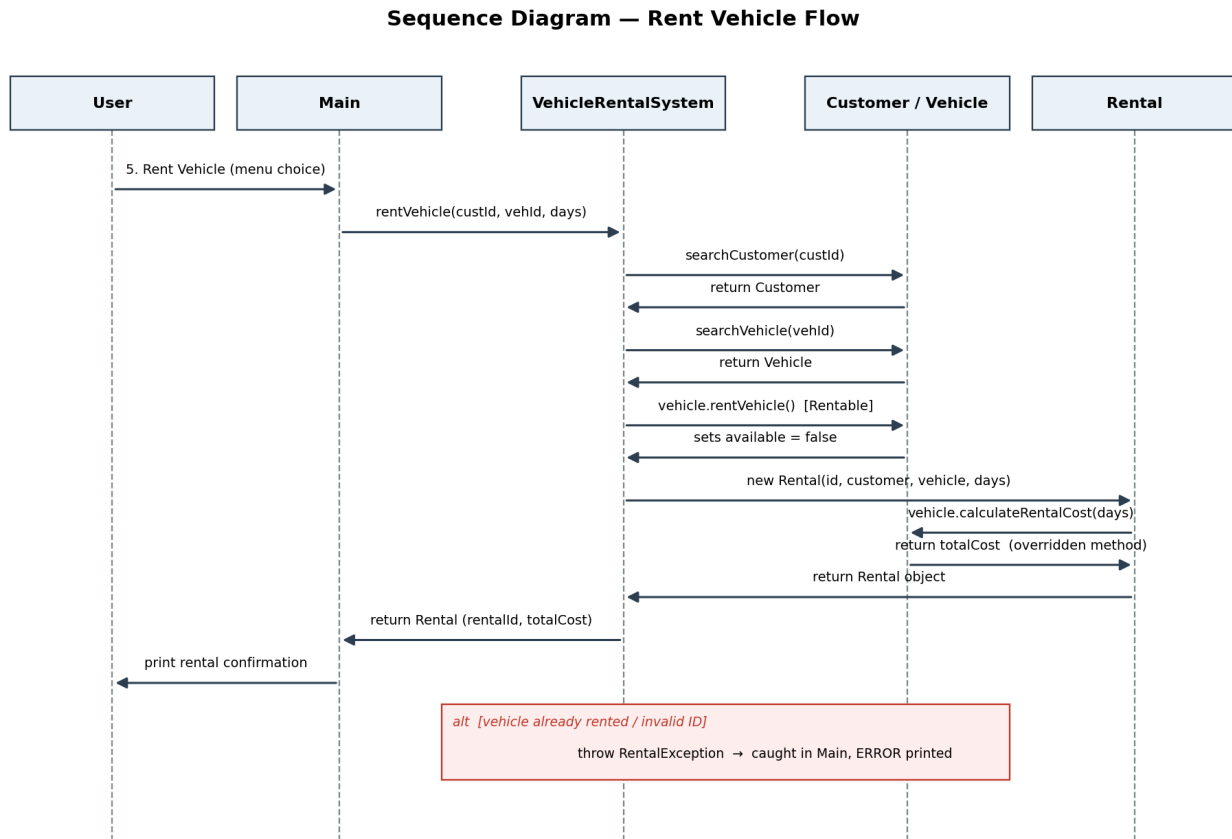


Figure 6.3 — Sequence Diagram for the Rent Vehicle flow.

6.4 Class / Component Diagram

The class diagram shows the full static structure of the system: the Rentable interface, the abstract Vehicle parent and its three concrete subclasses, the Customer and Rental classes, the RentalException, and how VehicleRentalSystem and Main sit above them as the logic and presentation layers.

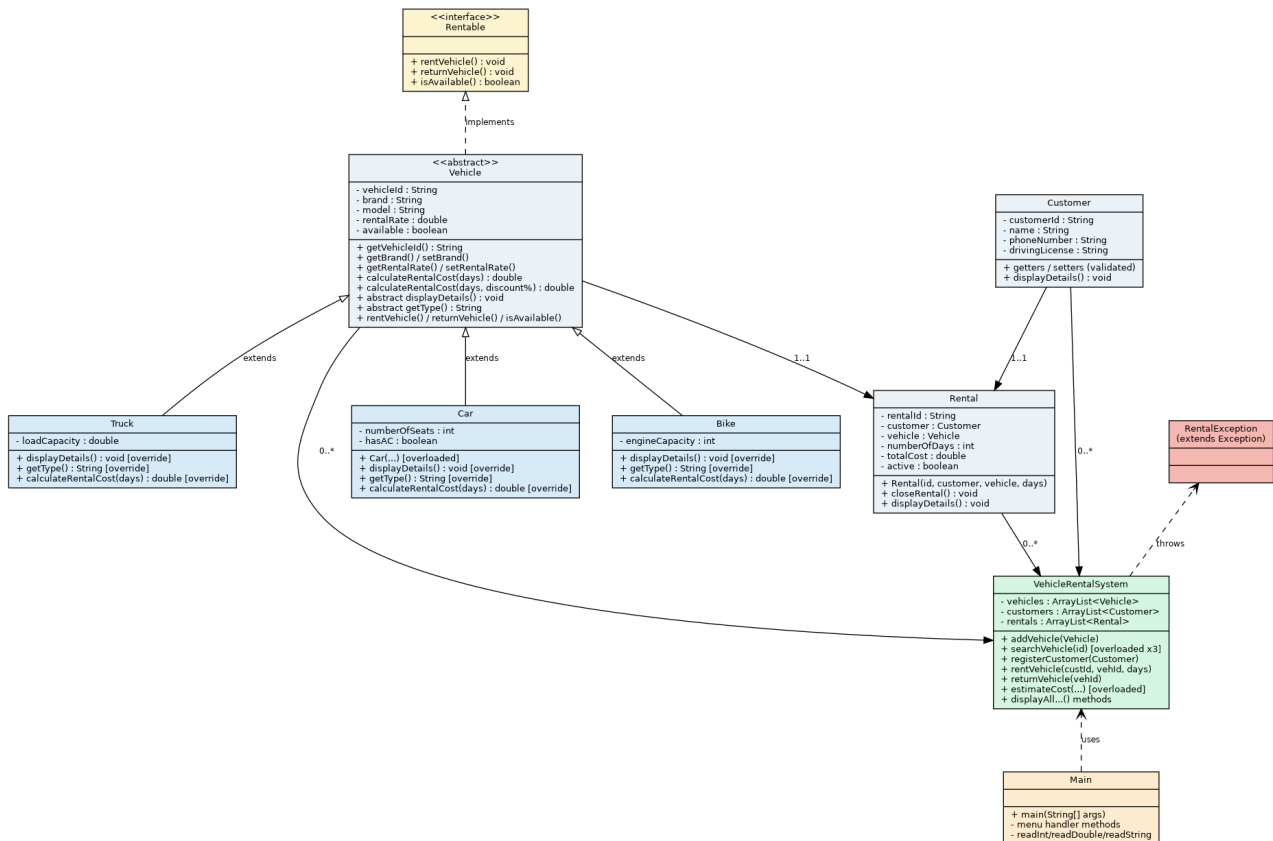


Figure 6.4 — Class Diagram showing inheritance, interface realisation, composition, and aggregation relationships.

6.5 ER Diagram

Not applicable. This project intentionally uses in-memory `ArrayList` collections rather than a database or file-based persistence layer (see Section 2, Scope, and Section 13, Future Enhancements), so there are no persisted tables or relations to model with an Entity-Relationship diagram. The closest analogue — how `Vehicle`, `Customer`, and `Rental` relate to one another at run time — is already captured by the composition/aggregation relationships in the Class Diagram above (Figure 6.4). Should a database layer be added in a future version, `Vehicle`, `Customer`, and `Rental` would map naturally onto three related tables, with `Rental` holding foreign keys to `Vehicle` and `Customer`.

7. Design Decisions & Rationale

Vehicle as an abstract class, not an interface

A generic "vehicle" has real shared state (ID, brand, model, rate, availability) and shared, non-trivial logic (cost calculation base formula, availability toggling). An interface cannot hold instance fields or constructors, so an abstract class was the correct tool — it lets every subclass inherit both data and behaviour while still forcing each one to define its own `displayDetails()` and `getType()`.

A separate `Rentable` interface alongside the abstract class

Renting/returning is a *capability* that, in principle, other kinds of items (e.g. equipment, trailers) could also have, independent of being a `Vehicle`. Modelling it as an interface keeps that behaviour decoupled from the `Vehicle` hierarchy and demonstrates the interface-implementation OOP requirement cleanly.

Pricing rules implemented via method overriding, not `if/else` on type

Each subclass overrides `calculateRentalCost(int)` and adds its own surcharge (AC charge for cars, helmet charge for bikes, load surcharge for trucks) by first calling `super.calculateRentalCost(days)` and then adding its own term. This avoids a long `if (type == "Car") ... else if (type == "Bike")` chain in the business logic layer, and lets `calculateRentalCost()` be extended for a new vehicle type without touching `VehicleRentalSystem` at all.

A custom checked exception (`RentalException`) instead of return codes

Using exceptions instead of boolean return values or error codes keeps the calling code (Main) free of repetitive `if-failed-then` checks, forces every business-rule violation to be handled explicitly (checked exceptions are enforced by the compiler), and keeps the success-path code readable.

Three `ArrayList`s in one class rather than one class per collection

Vehicles, customers, and rentals are closely related (a `Rental` references one `Customer` and one `Vehicle`) and are almost always operated on together, so keeping them in a single `VehicleRentalSystem` "facade" class was simpler for a project of this size than introducing separate repository classes, while still keeping Main free of any direct collection access.

Overloading used for both cost calculation and search

`calculateRentalCost(days)` / `calculateRentalCost(days, discount)` and the three `searchVehicle(...)` variants were chosen specifically to demonstrate compile-time polymorphism (overloading) alongside the runtime polymorphism already shown by method overriding, so the project covers both forms explicitly rather than only one.

In-memory storage only (`ArrayList`, no file/database persistence)

Persistence was deliberately left out to keep the project's scope appropriate for a second-year OOP assignment and to keep the focus on OOP concepts rather than I/O or SQL. The trade-off — all data is lost when the program exits — is documented as a known limitation and a natural next step (Section 13).

8. Implementation Details

8.1 Technology Stack

Component	Choice
Language	Java (core language only; developed and tested on JDK 21, compatible with JDK 8+)
Libraries	java.util.Scanner (console input), java.util.ArrayList (collections) — Java standard library only
Build / Run	javac (compiler), java (runtime) — no build tool (Maven/Gradle) required
Interface	Command-line / console (text-based menu)
Excluded by design	No database, no GUI framework, no Spring Boot, no third-party dependencies

8.2 File Structure

The project is organised as ten source files, one class/interface per file, all in the default package:

File	Role
Main.java	Entry point; console menu and input handling
VehicleRentalSystem.java	Core logic layer; owns the ArrayLists
Vehicle.java	Abstract parent class; implements Rentable
Car.java / Bike.java / Truck.java	Concrete vehicle subclasses
Customer.java	Encapsulated customer record
Rental.java	Links a Customer + Vehicle + duration + cost
Rentable.java	Interface: rentVehicle(), returnVehicle(), isAvailable()
RentalException.java	Custom checked exception for business-rule failures

8.3 How Each OOP Concept Was Implemented

Concept	Where / How
Encapsulation	All fields in Vehicle, Customer, and Rental are private; access is only through public getters and validated setters (e.g. setRentalRate() rejects values ≤ 0 ; setPhoneNumber() rejects numbers that are not 10 digits).
Inheritance	Car, Bike, and Truck extend Vehicle using the extends keyword and call super(...) in their constructors to initialise inherited fields.
Abstraction	Vehicle is declared abstract with two abstract methods, displayDetails() and getType(), so it cannot be instantiated directly and every subclass must supply its own implementation.

Concept	Where / How
Polymorphism	VehicleRentalSystem stores an ArrayList<Vehicle> that holds Car, Bike, and Truck objects interchangeably. Calling v.displayDetails() or v.calculateRentalCost(days) on a Vehicle reference executes the correct subclass version at run time (dynamic method dispatch).
Method Overloading	calculateRentalCost(int) / calculateRentalCost(int, double); three searchVehicle(...) signatures; two estimateCost(...) signatures; two Car constructors chained with this(...).
Method Overriding	displayDetails(), getType(), and calculateRentalCost() are overridden in Car, Bike, and Truck; toString() is overridden from java.lang.Object in Vehicle, Customer, and Rental.
Constructors	Every class defines an explicit constructor; child classes call super(...) as the first statement; Car additionally demonstrates constructor chaining with this(...).
Interfaces	Rentable declares rentVehicle(), returnVehicle(), and isAvailable(); Vehicle implements it, so all three subclasses inherit the implementation.
Collections	Three ArrayList<T> fields (vehicles, customers, rentals) in VehicleRentalSystem, iterated with the enhanced for-loop throughout.
Exception Handling	RentalException (a checked exception extending Exception) is thrown for every business-rule violation and caught in Main with try/catch/finally; NumberFormatException is caught separately for invalid numeric input.

8.4 Key Code Excerpts

Abstraction — abstract class with abstract methods (Vehicle.java)

```
public abstract class Vehicle implements Rentable {
    private String vehicleId, brand, model;
    private double rentalRate;
    private boolean available;

    public abstract void displayDetails();
    public abstract String getType();

    public double calculateRentalCost(int days) {
        return rentalRate * days;
    }
}
```

Overriding + polymorphic pricing rule (Car.java)

```
public class Car extends Vehicle {
    private int numberOfSeats;
    private boolean hasAC;

    @Override
    public double calculateRentalCost(int days) {
        double cost = super.calculateRentalCost(days);
        if (hasAC) { cost += 200 * days; }
        return cost;
    }
}
```

Interface + custom exception (Vehicle.java)

```
public void rentVehicle() throws RentalException {
    if (!available) {
        throw new RentalException("Vehicle " + vehicleId + " is already rented.");
    }
    available = false;
}
```

Polymorphism in action (VehicleRentalSystem.displayAllVehicles)

```
for (Vehicle v : vehicles) {
    v.displayDetails(); // runs Car/Bike/Truck's own version
}
```

9. Screenshots / Results

The screenshots below were captured from an actual run of the compiled program (javac *.java && java Main), confirming that the application builds and behaves as designed.

9.1 Exception Handling in Practice

The screenshot below is taken directly from a real test run on the developer's machine. The operator attempted to add a Bike with vehicle ID V101, which already exists in the preloaded sample data. Instead of crashing or silently overwriting the existing record, the system caught the condition inside `addVehicle()`, threw a `RentalException`, and `Main` printed a clean, readable error message before returning to the menu — exactly the behaviour required by FR-1 and FR-12.

```
===== VEHICLE RENTAL SYSTEM =====
1. Add Vehicle
2. Display Vehicles
3. Search Vehicle
4. Register Customer
5. Rent Vehicle
6. Return Vehicle
7. Calculate Rental Cost
8. Display Customer Details
9. Display Rental Details
0. Exit
=====
Enter your choice: 1
Vehicle type -> 1.Car 2.Bike 3.Truck
Choose type: 2
Vehicle ID   : V101
Brand       :      Maruti Swift
Model      :      Maruti Swift
Rent per day : 1500
Engine (cc) : 650
ERROR: Vehicle ID V101 already exists.

===== VEHICLE RENTAL SYSTEM =====
1. Add Vehicle
2. Display Vehicles
3. Search Vehicle
4. Register Customer
5. Rent Vehicle
6. Return Vehicle
7. Calculate Rental Cost
8. Display Customer Details
9. Display Rental Details
```

Figure 9.1 — Real console output showing duplicate-ID validation (`RentalException` caught and reported).

9.2 Displaying the Fleet (Polymorphism)

Choosing option 2 (Display Vehicles) prints every vehicle by calling `displayDetails()` through a `Vehicle` reference. Note how each row shows a different extra field — Seats/AC for cars, Engine for the bike, Load for the truck — proving that the subclass-specific override is being invoked at run time for every object in the same `ArrayList<Vehicle>`.

```
Vehicle Rental System - Display Vehicles

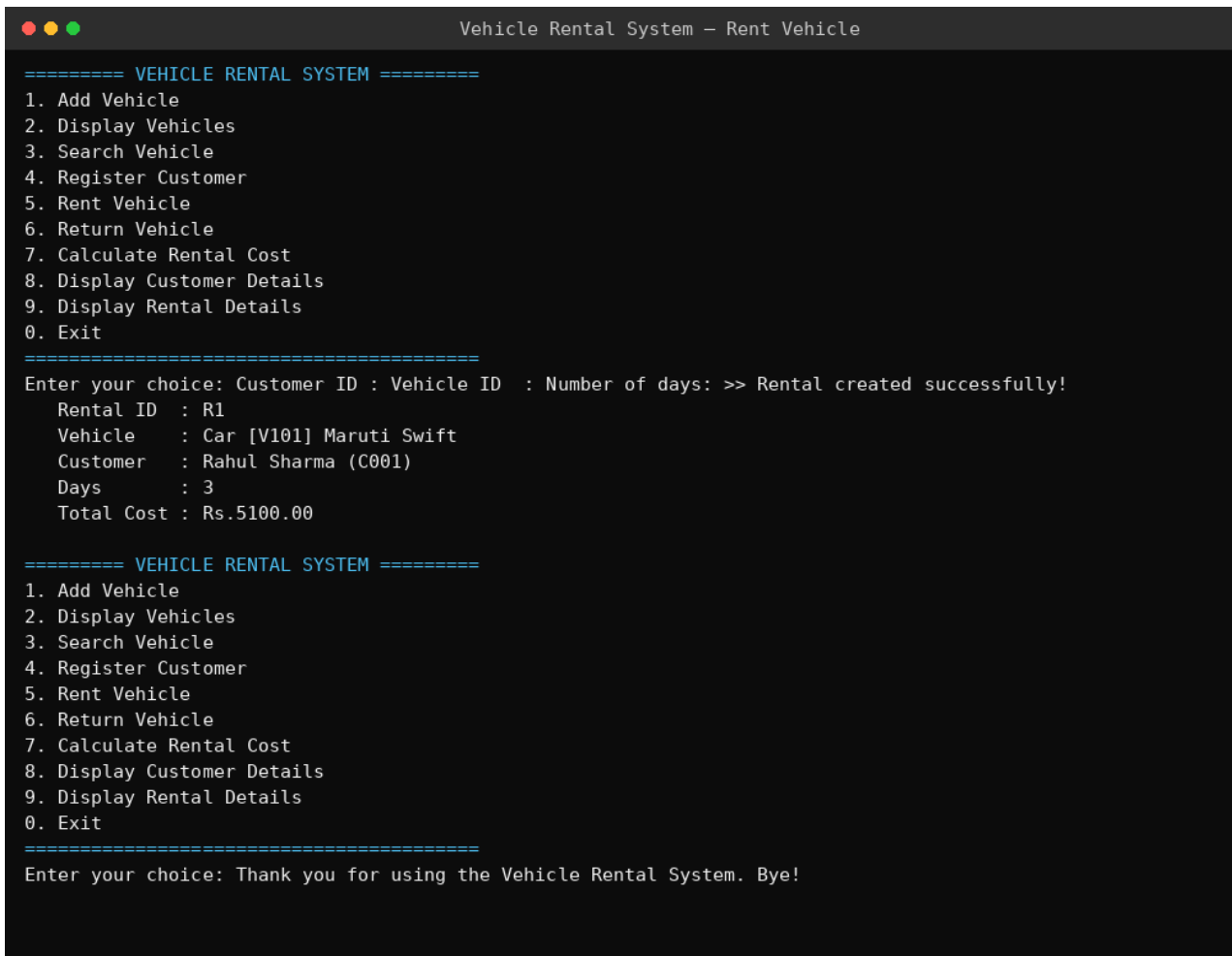
===== VEHICLE RENTAL SYSTEM =====
1. Add Vehicle
2. Display Vehicles
3. Search Vehicle
4. Register Customer
5. Rent Vehicle
6. Return Vehicle
7. Calculate Rental Cost
8. Display Customer Details
9. Display Rental Details
0. Exit
=====
Enter your choice: 1. All vehicles    2. Only available vehicles
Choose: ID      | Type      | Brand      | Model      | Rate/Day    | Status
-----
V101 | Car       | Maruti     | Swift      | Rs.1500.00  | Available | Seats: 5 | AC: Yes
V102 | Car       | Hyundai    | i20        | Rs.1800.00  | Available | Seats: 5 | AC: No
V201 | Bike      | Honda      | Shine      | Rs.400.00   | Available | Engine: 125 cc
V301 | Truck     | Tata       | 407        | Rs.3000.00  | Available | Load: 2.5 tons

===== VEHICLE RENTAL SYSTEM =====
1. Add Vehicle
2. Display Vehicles
3. Search Vehicle
4. Register Customer
5. Rent Vehicle
6. Return Vehicle
7. Calculate Rental Cost
8. Display Customer Details
9. Display Rental Details
0. Exit
=====
Enter your choice: Thank you for using the Vehicle Rental System. Bye!
```

Figure 9.2 — Fleet listing showing Car, Car, Bike, and Truck rows, each formatted by its own overridden `displayDetails()`.

9.3 Renting a Vehicle

Renting vehicle V101 (a Car with AC) to customer C001 for 3 days produces a total cost of Rs.5100.00, matching the formula $1500 \times 3 + 200 \times 3 = 4500 + 600 = 5100$.



```
Vehicle Rental System - Rent Vehicle

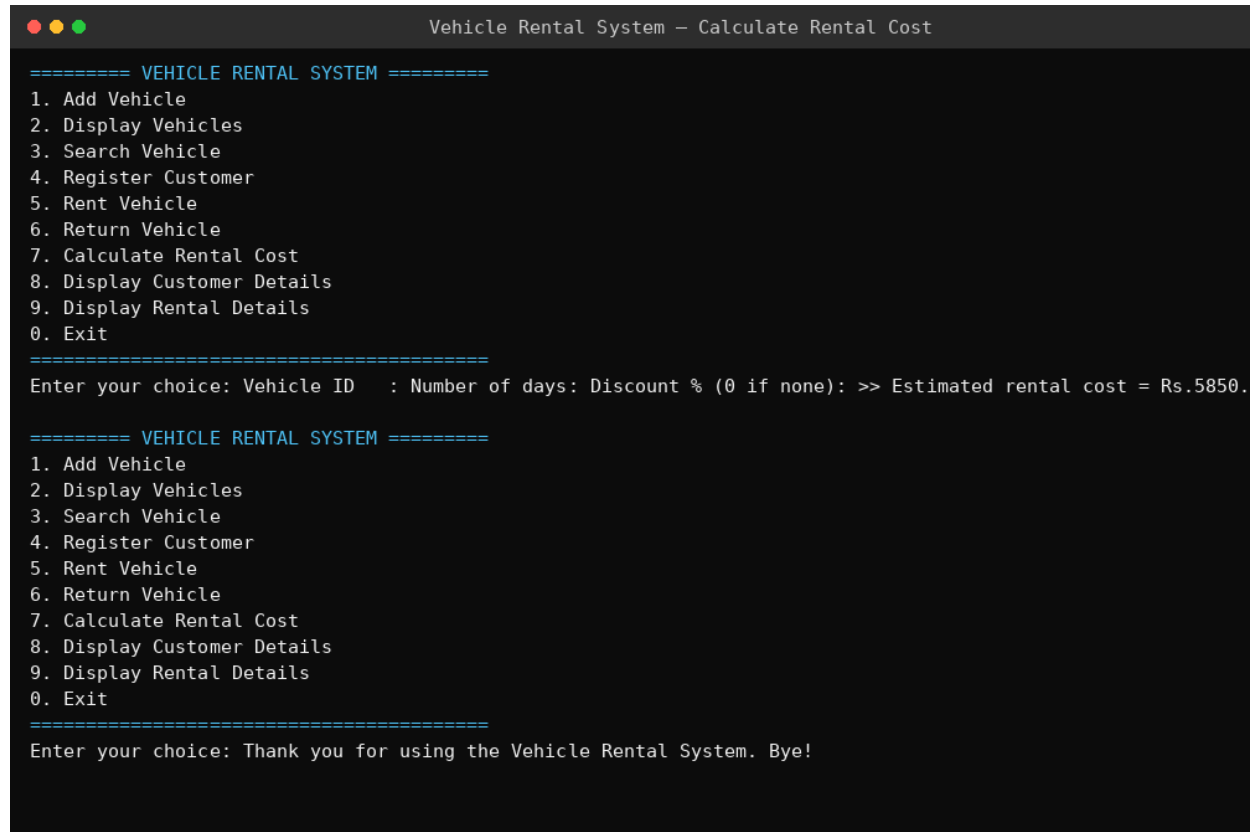
===== VEHICLE RENTAL SYSTEM =====
1. Add Vehicle
2. Display Vehicles
3. Search Vehicle
4. Register Customer
5. Rent Vehicle
6. Return Vehicle
7. Calculate Rental Cost
8. Display Customer Details
9. Display Rental Details
0. Exit
=====
Enter your choice: Customer ID : Vehicle ID : Number of days: >> Rental created successfully!
  Rental ID   : R1
  Vehicle     : Car [V101] Maruti Swift
  Customer    : Rahul Sharma (C001)
  Days        : 3
  Total Cost  : Rs.5100.00

===== VEHICLE RENTAL SYSTEM =====
1. Add Vehicle
2. Display Vehicles
3. Search Vehicle
4. Register Customer
5. Rent Vehicle
6. Return Vehicle
7. Calculate Rental Cost
8. Display Customer Details
9. Display Rental Details
0. Exit
=====
Enter your choice: Thank you for using the Vehicle Rental System. Bye!
```

Figure 9.3 — Successful rental transaction with automatically calculated cost.

9.4 Discounted Cost Estimate (Overloading)

Estimating the cost of truck V301 for 2 days with a 10% discount uses the second, overloaded `estimateCost(vehicleId, days, discount)` method: base cost ($3000 \times 2 + 100 \times 2.5 \times 2 = 6500$) less a 10% discount = Rs.5850.00.



```
Vehicle Rental System - Calculate Rental Cost

===== VEHICLE RENTAL SYSTEM =====
1. Add Vehicle
2. Display Vehicles
3. Search Vehicle
4. Register Customer
5. Rent Vehicle
6. Return Vehicle
7. Calculate Rental Cost
8. Display Customer Details
9. Display Rental Details
0. Exit
=====
Enter your choice: Vehicle ID : Number of days: Discount % (0 if none): >> Estimated rental cost = Rs.5850.

===== VEHICLE RENTAL SYSTEM =====
1. Add Vehicle
2. Display Vehicles
3. Search Vehicle
4. Register Customer
5. Rent Vehicle
6. Return Vehicle
7. Calculate Rental Cost
8. Display Customer Details
9. Display Rental Details
0. Exit
=====
Enter your choice: Thank you for using the Vehicle Rental System. Bye!
```

Figure 9.4 — Discounted rental cost calculated via method overloading.

9.5 Returning a Vehicle and Viewing Rental History

After renting and then returning vehicle V101, the rental record is marked RETURNED and remains visible in the rental history, demonstrating that the Rental object's active flag and the underlying Vehicle's availability stay in sync.

```
Vehicle Rental System – Return Vehicle & Rental Records

===== VEHICLE RENTAL SYSTEM =====
1. Add Vehicle
2. Display Vehicles
3. Search Vehicle
4. Register Customer
5. Rent Vehicle
6. Return Vehicle
7. Calculate Rental Cost
8. Display Customer Details
9. Display Rental Details
0. Exit

=====
Enter your choice: Customer ID : Vehicle ID : Number of days: >> Rental created successfully!
Rental ID : R1
Vehicle : Car [V101] Maruti Swift
Customer : Rahul Sharma (C001)
Days : 3
Total Cost : Rs.5100.00

===== VEHICLE RENTAL SYSTEM =====
1. Add Vehicle
2. Display Vehicles
3. Search Vehicle
4. Register Customer
5. Rent Vehicle
6. Return Vehicle
7. Calculate Rental Cost
8. Display Customer Details
9. Display Rental Details
0. Exit

=====
Enter your choice: Vehicle ID to return: >> Vehicle returned successfully.
Rental R1 closed. Amount payable: Rs.5100.00

===== VEHICLE RENTAL SYSTEM =====
1. Add Vehicle
2. Display Vehicles
3. Search Vehicle
4. Register Customer
5. Rent Vehicle
6. Return Vehicle
7. Calculate Rental Cost
8. Display Customer Details
9. Display Rental Details
0. Exit

=====
Enter your choice: ID | Customer | Vehicle | Days | Total Cost | Status
-----
R1 | Rahul Sharma (C001) | Car [V101] Maruti Swift | 3 | Rs.5100.00 | RETURNED

===== VEHICLE RENTAL SYSTEM =====
1. Add Vehicle
2. Display Vehicles
3. Search Vehicle
4. Register Customer
5. Rent Vehicle
6. Return Vehicle
7. Calculate Rental Cost
8. Display Customer Details
9. Display Rental Details
0. Exit

=====
Enter your choice: Thank you for using the Vehicle Rental System. Bye!
```

Figure 9.5 — Return flow and the resulting rental history table.

10. Testing Approach

Testing was performed manually by walking through the menu and comparing printed output against the expected result for each scenario, since the project's console-driven nature and small scope did not warrant a separate automated test framework. Both interactive typing and scripted, piped input (for repeatable regression checks) were used.

#	Test Case	Steps	Expected Result	Status
T1	Preloaded sample data	Start program; choose 2 → 1	4 vehicles (V101–V301) shown as Available	Pass
T2	Polymorphic display	Choose 2 → 1	Each vehicle row shows its own type-specific field (seats/AC, cc, tons)	Pass
T3	Rent a vehicle	Choose 5; customer C001, vehicle V101, 3 days	Total cost Rs.5100.00; vehicle becomes unavailable	Pass
T4	Prevent double renting	Repeat T3 immediately	ERROR: Vehicle V101 is already rented.	Pass
T5	Overloaded cost estimate	Choose 7; vehicle V301, 2 days, 10% discount	Rs.5850.00 (vs. Rs.6500.00 with 0% discount)	Pass
T6	Overloaded search (by ID)	Choose 3 → 1; ID V201	Bike details returned	Pass
T7	Overloaded search (brand+model)	Choose 3 → 2; Maruti / Swift	Matching car(s) returned	Pass
T8	Overloaded search (max rate)	Choose 3 → 3; max Rs.500	Only the bike (Rs.400/day) returned	Pass
T9	Return flow	Choose 6; vehicle V101	Rental closed; amount payable shown; vehicle available again	Pass
T10	Prevent invalid return	Repeat T9 immediately	ERROR: No ongoing rental found for vehicle V101	Pass
T11	Duplicate vehicle ID rejected	Choose 1; enter existing ID V101	ERROR: Vehicle ID V101 already exists. (see Figure 9.1)	Pass
T12	Invalid menu choice	Enter 15 at the main menu	ERROR: Invalid choice. Please pick a number from 0 to 9.	Pass
T13	Non-numeric input	Enter "abc" where a number is expected	"!! Please enter a valid whole number." then re-prompt (no crash)	Pass
T14	Zero/negative rental days	Choose 5; enter 0 days	ERROR: Number of days must be at least 1.	Pass
T15	Unknown customer/vehicle ID	Choose 5; enter customer C999	ERROR: No customer found with ID: C999	Pass

All fifteen test cases passed on JDK 21 after compiling with `javac *.java`. The program did not terminate abnormally (no uncaught exception / stack trace) under any of the above scenarios, confirming the exception-handling requirement (FR-11, FR-12) is met in practice.

11. Challenges Faced

Choosing between an abstract class and an interface for Vehicle

Early on it was tempting to make Vehicle an interface so any class could "be a vehicle." This was rejected because Vehicle needed genuine shared state (ID, brand, rate, availability) and partially shared behaviour (the base cost formula), which interfaces cannot hold. The resolution was to combine both: an abstract class for shared state/behaviour, plus a separate Rentable interface for the renting capability specifically.

Keeping pricing logic out of the business-logic layer

An early draft calculated the AC/helmet/load surcharges inside VehicleRentalSystem using if/else checks on vehicle type, which defeated the purpose of polymorphism and would not scale to new vehicle types. Moving each surcharge into the relevant subclass's overridden calculateRentalCost() (calling super for the base amount) fixed this and made the design open for extension without modifying existing code.

Avoiding a crash on bad console input

Scanner.nextInt() throws an unchecked InputMismatchException on non-numeric input and leaves a leftover newline in the buffer if used carelessly, which can silently break subsequent reads. The fix was to always read a full line with sc.nextLine() and parse it manually inside a try/catch for NumberFormatException, re-prompting on failure.

Distinguishing checked exceptions from validation logic

It took some iteration to decide which failures should be enforced by the compiler (checked RentalException, e.g. renting an unavailable vehicle) versus which were purely input-format issues handled locally (NumberFormatException from Integer.parseInt). Separating these two concerns made both the method signatures and the error messages clearer.

Keeping vehicle availability and rental status in sync

Because availability lives on Vehicle and the active/returned flag lives on Rental, there was a risk of the two falling out of sync (e.g. a vehicle marked available while its rental was still open). This was solved by only ever changing availability through the Rentable interface methods (rentVehicle() / returnVehicle()), which are always called together with the corresponding Rental creation/closure inside VehicleRentalSystem — never directly by Main.

12. Learnings & Key Takeaways

- **Abstraction and inheritance work best together** when the parent class carries real shared state and only the type-specific behaviour is left abstract — this avoids both code duplication and forced, meaningless implementations.
- **Polymorphism is most valuable when it replaces conditional logic.** Overriding `calculateRentalCost()` in each subclass removed what would otherwise have been a growing if/else chain in the business-logic layer, and it means adding a new vehicle type never requires touching `VehicleRentalSystem`.
- **Interfaces and abstract classes solve different problems** — an interface defines a capability/contract (`Rentable`), while an abstract class defines a partially-implemented "is-a" relationship (`Vehicle`). Using both together, rather than picking one, produced a cleaner design than either alone.
- **Encapsulation is not just about hiding fields** — the real value showed up in validated setters (e.g. rejecting a negative rental rate or a malformed phone number), which is only possible because the fields were private in the first place.
- **Checked exceptions enforce discipline.** Because `RentalException` must be declared or caught, it was impossible to forget to handle a failed rental or return somewhere in the code — the compiler caught any omission immediately.
- **Layered design pays off even in a small project.** Keeping `Main` free of data storage and `VehicleRentalSystem` free of console I/O made the code far easier to test, reason about, and would make it straightforward to later swap the console UI for a GUI without touching the business logic at all.
- **Manual testing benefits from scripted input.** Piping input via `echo/printf` into java `Main` made it possible to re-run the exact same multi-step scenario repeatedly while iterating on the code, rather than retyping each test by hand.

13. Future Enhancements

- **Persistent storage** — save vehicles, customers, and rentals to a file (serialization/CSV) or a database so data survives after the program exits, which would also make an ER diagram directly applicable.
- **Late-return penalties** — track an actual due date using `java.time.LocalDate` and add a surcharge for overdue returns.
- **Faster lookups** — replace the current linear `ArrayList` search with a `HashMap<String, Vehicle>` / `HashMap<String, Customer>` keyed by ID for $O(1)$ lookups as the fleet grows.
- **A second `Rentable` implementer** — add a non-vehicle rentable item (e.g. equipment or a trailer) to demonstrate that the interface is genuinely reusable beyond the `Vehicle` hierarchy.
- **Automated unit tests** — add JUnit test cases for the cost-calculation and validation logic to complement the manual testing in Section 10.
- **User roles / authentication** — separate admin and staff logins with different permissions.

- **A graphical or web front end** — reuse the existing VehicleRentalSystem logic layer unchanged behind a GUI (Swing/JavaFX) or a web interface, since the presentation layer was deliberately kept independent of the business logic.

14. References

- 1 Oracle. *The Java Tutorials* — "Object-Oriented Programming Concepts."
<https://docs.oracle.com/javase/tutorial/java/concepts/>
- 2 Oracle. *The Java Tutorials* — "Classes and Objects."
<https://docs.oracle.com/javase/tutorial/java/javaOO/>
- 3 Oracle. *Java Platform, Standard Edition Documentation* — `java.util.ArrayList`.
<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/ArrayList.html>
- 4 Oracle. *The Java Tutorials* — "Exceptions."
<https://docs.oracle.com/javase/tutorial/essential/exceptions/>
- 5 Oracle. *The Java Tutorials* — "Interfaces and Inheritance."
<https://docs.oracle.com/javase/tutorial/java/landI/>
- 6 GeeksforGeeks. "Object Oriented Programming (OOPs) Concept in Java."
<https://www.geeksforgeeks.org/java/object-oriented-programming-oops-concept-in-java/>
- 7 Course lecture notes and lab manual for Object-Oriented Programming with Java (institution-provided).

End of Report